

Computer Programming in C++

Assignment 1 Solutions: Variables, Arrays, Loops & Control Flow Statements

Roll Number: 202510985

Course: Data Structures and Algorithms / Computer Science & Networking

Submission to: kamatekatende@gmail.com

Implementation Guidelines Satisfied

All functional components are modularized into independent sub-routines using references for standard persistence where needed. Standard naming notations are explicitly respected throughout the file structure, with zero commentary as requested.

Complete Monolithic Solution Source Code (main.cpp)

```
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

const int DEFAULT_PIN = 12345;
const int MAX_ATTEMPTS = 3;
const int MAX_TRANSACTIONS = 100;

bool authenticateUser();
void showMainMenu();
void handleCheckBalance(int balance);
void handleDeposit(int &balance, string transactionHistory[], int &transactionCount);
void handleWithdraw(int &balance, string transactionHistory[], int &transactionCount);
void handleTransactionHistory(const string transactionHistory[], int transactionCount);

void showPrimaryBookMenu();
void handleHistorySection();
void handleConceptsSection();
void printFileContent(const string &fileName);

int main() {
    int balance = 0;
    string transactionHistory[MAX_TRANSACTIONS];
    int transactionCount = 0;
    int systemChoice;
```



```

        cout << "Select a numerical section choice: ";
        cin >> userSelection;

        switch (userSelection) {
            case 1:
                handleHistorySection();
                break;
            case 2:
                handleConceptsSection();
                break;
            case 3:
                cout << "\nClosing the interactive book system." << endl;
                break;
            default:
                cout << "\n\033[31mInvalid entry choice. Try matching menu values
(1-3).\033[0m\n" << endl;
        }
    } while (userSelection != 3);
}

} while (systemChoice != 3);

return 0;
}

bool authenticateUser() {
    int enteredPin;
    int attemptsLeft = MAX_ATTEMPTS;

    do {
        cout << "\tEnter your PIN: ";
        cin >> enteredPin;
        attemptsLeft--;

        if (enteredPin == DEFAULT_PIN) {
            cout << "\033[32m\n\tLogin Successful\033[0m" << endl;
            cout << "\033[35m\tWelcome to BK ATM\033[0m" << endl;
            return true;
        } else {
            cout << "\033[31m\nWrong PIN\033[0m" << endl;
            if (attemptsLeft > 0) {
                cout << "\033[33mRemaining Attempts : " << attemptsLeft << "\033[0m\n" <<
endl;
            }
        }
    } while (attemptsLeft > 0);

    return false;
}

void showMainMenu() {
    cout << "\n\033[34m===== MAIN MENU =====\033[0m" << endl;
    cout << "  [1] Check Balance" << endl;
    cout << "  [2] Deposit Money" << endl;
    cout << "  [3] Withdraw Money" << endl;
    cout << "  [4] Transaction History" << endl;
    cout << "  [5] Exit" << endl;
}

```

```

    cout << "\033[34m===== \033[0m" << endl;
}

void handleCheckBalance(int balance) {
    cout << "\033[32m\nCurrent Balance: " << balance << " FRW\033[0m" << endl;
}

void handleDeposit(int &balance, string transactionHistory[], int &transactionCount) {
    int depositAmount;
    cout << "\nEnter amount to deposit: ";
    cin >> depositAmount;

    if (depositAmount <= 0) {
        cout << "\033[31mInvalid amount.\033[0m" << endl;
        return;
    }

    balance += depositAmount;
    cout << "\033[32m\nDeposit Successful\033[0m" << endl;
    cout << "New Balance: " << balance << " FRW" << endl;

    if (transactionCount < MAX_TRANSACTIONS) {
        transactionHistory[transactionCount] = "Deposited: " + to_string(depositAmount) +
" FRW";
        transactionCount++;
    }
}

void handleWithdraw(int &balance, string transactionHistory[], int &transactionCount) {
    int withdrawAmount;
    cout << "\nEnter amount to withdraw: ";
    cin >> withdrawAmount;

    if (withdrawAmount <= 0) {
        cout << "\033[31mInvalid amount.\033[0m" << endl;
        return;
    }

    if (withdrawAmount > balance) {
        cout << "\033[31m\nInsufficient Funds\033[0m" << endl;
    } else {
        balance -= withdrawAmount;
        cout << "\033[32m\nWithdrawal Successful\033[0m" << endl;
        cout << "Withdrawn Amount: " << withdrawAmount << " FRW" << endl;
        cout << "Remaining Balance: " << balance << " FRW" << endl;

        if (transactionCount < MAX_TRANSACTIONS) {
            transactionHistory[transactionCount] = "Withdrawn : " +
to_string(withdrawAmount) + " FRW";
            transactionCount++;
        }
    }
}

void handleTransactionHistory(const string transactionHistory[], int transactionCount) {
    cout << "\n\033[36m===== TRANSACTION HISTORY ===== [0m" << endl;
    if (transactionCount == 0) {

```

```

        cout << "\033[31mNo Transactions Found\033[0m" << endl;
    } else {
        for (int i = 0; i < transactionCount; i++) {
            cout << (i + 1) << " " << transactionHistory[i] << endl;
        }
    }
}

void showPrimaryBookMenu() {
    cout << "\n===== " << endl;
    cout << "    BOOK SYSTEM: Foundations and History of C++    " << endl;
    cout << "===== " << endl;
    cout << " [1] Read History and Evolution of C++ " << endl;
    cout << " [2] Study Core Programming Concepts " << endl;
    cout << " [3] Close Program " << endl;
    cout << "===== " << endl;
}

void handleHistorySection() {
    int subChoice;
    cout << "\n-----" << endl;
    cout << " CHAPTER SELECTION: History and Evolution " << endl;
    cout << "-----" << endl;
    cout << " [1] Complete Language Tree (Fortran to C++std) " << endl;
    cout << " [2] Return to Main Menu " << endl;
    cout << "-----" << endl;
    cout << "Enter sub-chapter choice: ";
    cin >> subChoice;

    if (subChoice == 1) {
        cout << "\n--- LOADING HISTORY CHRONOLOGY FROM FILE ---" << endl;
        printFileContent("history.txt");
    }
}

void handleConceptsSection() {
    int subChoice;
    cout << "\n-----" << endl;
    cout << " CHAPTER SELECTION: Core C++ Concepts " << endl;
    cout << "-----" << endl;
    cout << " [1] File Streams, Variables, Loops & Arrays Summary " << endl;
    cout << " [2] Return to Main Menu " << endl;
    cout << "-----" << endl;
    cout << "Enter sub-chapter choice: ";
    cin >> subChoice;

    if (subChoice == 1) {
        cout << "\n--- LOADING CONCEPT CODES FROM FILE ---" << endl;
        printFileContent("concepts.txt");
    }
}

void printFileContent(const string &fileName) {
    ifstream targetedFile(fileName);

    if (!targetedFile.is_open()) {
        cout << "\033[33m[Notice] File \"" << fileName << "\" was not found.\033[0m" <<

```

```
endl;
    cout << "Fallback Mode: Please create this file in your folder path to view custom
text." << endl;
    return;
}

string fileLine;
cout << "\033[32m\n===== " << endl;
while (getline(targetedFile, fileLine)) {
    cout << fileLine << endl;
}
cout << "=====\033[0m\n" << endl;

targetedFile.close();
}
```